Homework — 2.3.1 Algorithms — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. - (a) **[1]** — **O(log n)** (this is binary search). - (b) **[1]** — Each step **halves** the remaining search space, so **doubling n adds only one extra step**; the number of steps grows with $\log_2$ n, giving O(log n).

2. [3] — All four pass rows correct = 3 marks; three rows correct = 2; two rows correct = 1. Note Pass 4 still performs a swap (2 ↔ 1) — the list is only fully sorted after Pass 4.

Start: `[6, 2, 9, 4, 1]`

After pass	Result
Pass 1	`[2, 6, 4, 1, 9]`
Pass 2	`[2, 4, 1, 6, 9]`
Pass 3	`[2, 1, 4, 6, 9]`
Pass 4	`[1, 2, 4, 6, 9]`

Working — Pass 1: 6,2→swap `[2,6,9,4,1]`; 6,9→no; 9,4→swap `[2,6,4,9,1]`; 9,1→swap `[2,6,4,1,9]`. Pass 2: 2,6→no; 6,4→swap `[2,4,6,1,9]`; 6,1→swap `[2,4,1,6,9]`; 6,9→no. Pass 3: 2,4→no; 4,1→swap `[2,1,4,6,9]`; 4,6→no; 6,9→no. Pass 4: 2,1→swap `[1,2,4,6,9]`; 2,4→no; 4,6→no; 6,9→no.

3. [3] — Binary search for **31** in `[4, 9, 15, 18, 23, 27, 31, 42, 56]`, `mid = (low + high) DIV 2`.

Step	low	high	mid (index)	value at mid	vs target 31
1	0	8	4	23	23 < 31 → low = mid + 1 = 5
2	5	8	6	31	equal → **found**

- 1 mark: Step 1 — mid = (0+8) DIV 2 = 4; value 23 < 31, so the lower half is discarded and low = 5.
- 1 mark: Step 2 — mid = (5+8) DIV 2 = 6; value 31 equals the target.
- 1 mark: result — **31 found at index 6**.

4. [3] — Tree: root 55; left subtree 32 (children 20, 45); right subtree 78 (children 60, 90). - (a) **[1]** — Depth-first (post-order), left → right → node: **20, 45, 32, 60, 90, 78, 55**. - (b) **[1]** — Breadth-first, level by level left-to-right: **55, 32, 78, 20, 45, 60, 90**. - (c) **[1]** — In-order traversal of a binary search tree outputs the values in **ascending (sorted) order** (here 20, 32, 45, 55, 60, 78, 90).

5. [3] — Dijkstra from A. Final shortest distances:

Node	Shortest distance from A	Via
A	0	—
B	2	A
C	3	B
D	6	C
E	8	D

Working: - A = 0. Tentative from A: B = 2, C = 5. Settle **B = 2 (via A)** (smallest tentative). - From B: C via B = 2 + 1 = **3** (< 5, update **C = 3 via B**); D via B = 2 + 7 = 9. Settle **C = 3 (via B)**. - From C: D via C = 3 + 3 = **6** (< 9, update **D = 6 via C**); E via C = 3 + 8 = 11. Settle **D = 6 (via C)**. - From D: E via D = 6 + 2 = **8** (< 11, update **E = 8 via D**). Settle **E = 8 (via D)**.

Award: all four distances correct (B=2, C=3, D=6, E=8) = 3 marks; two or three correct = 2; one correct = 1. The "via" column supports the working. Full credit only for the final settled distances — C and D must be the updated values (3 and 6), not the initial tentative values (5 and 9).

Part B (6 marks)

6. [2] — 1 mark: **intractable**. 1 mark: a valid example — brute-force Travelling Salesman Problem; trying every possible route/combination; brute-force password cracking (any genuinely exponential-or-worse problem).

7. [2] — 1 mark for one advantage: problems that are naturally self-similar (e.g. divide and conquer, tree traversal) can be expressed more simply / in less code with recursion. 1 mark for one disadvantage: each recursive call adds a **stack frame**, so recursion uses more memory and risks **stack overflow** (and call overhead can make it slower) compared with iteration.

8. [2] — 1 mark for one benefit: independent parts complete **at the same time**, so the overall task finishes faster / cores are used efficiently / the program stays responsive. 1 mark for one drawback: it is harder to design/test/debug; coordinating tasks adds **overhead**; shared data risks a **race condition**; on a single core there may be no real speed-up.

Part C (5 marks)

9. [5] — Indicative content; reward valid comparison points and a justified recommendation. Top band (4–5) requires a developed comparison that explicitly justifies the choice using the **all-destinations** nature of the task.

- Both find the **optimal (shortest) path** when all edge weights are **non-negative** (A *additionally requires a heuristic that never overestimates)* (1)*.
- **Dijkstra** uses only the actual cost so far and expands **outward in all directions**, naturally producing the shortest path from the source to **every** node *(1)*.
- **A*** uses **f(n) = g(n) + h(n)**, where h(n) is a heuristic estimate of the remaining cost to **a single goal**; this directs the search toward **one** target, so it is not designed to find all shortest paths at once *(1)*.
- Because the task needs the shortest distance to **every** delivery point, A *would have to be re-run once per destination and its heuristic gains little, whereas Dijkstra solves all destinations in a single run* (1)*.

- **Recommendation: Dijkstra's algorithm** — the single-source-to-all-nodes requirement matches exactly what Dijkstra computes; A *would only be preferable for a single known target with a good heuristic available* (1)*.

Total: 25 marks (Part A = 2 + 3 + 3 + 3 + 3 = 14, Part B = 6, Part C = 5).